

0.15.2. Deux rythmes, un seul chronomètre — doublé, deux positions

Les deux rythmes à deux places : en colonne 0 et en colonne 10.

1. Ce qu'il faut comprendre

C'est le 0.15 et le 0.15.1 réunis : les deux `poser` du 0.15, et les deux du 0.15.1.

Quatre `poser`, deux variables : la lente est la même en colonne 0 et en colonne 10, la rapide aussi.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1495 octets** de cartouche.

1. UNE VARIABLE — LENTE

```
uint8_t lente = 0;    // QUELLE lettre montre la lente : 0 = A ... 25 = Z
```

2. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;   // QUELLE lettre montre la rapide : 0 = A ... 25 = Z
```

3. UNE VARIABLE — IMAGES

```
uint8_t images = 0;   // UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();                                // attend l'image suivante : 60 tours par seconde

        // Le chronomètre avance d'une image, et repart à 0 après 59 :
        // il fait un tour complet chaque seconde.
        images++;
        if (images == 60) images = 0;

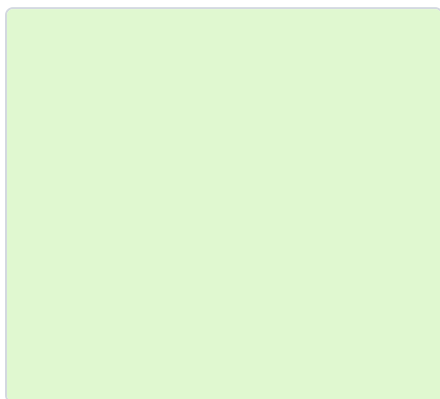
        // La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde).
        //  images % 15 = le reste de images divisé par 15 ;
        //  il vaut 0 seulement pour les multiples de 15.
        // (rapide + 1) % sizeof(ALPHABET) : la lettre suivante,
        //  et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A.
        if (images % 15 == 0) rapide = (rapide + 1) % sizeof(ALPHABET);

        // La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde).
        if (images == 0) lente = (lente + 1) % sizeof(ALPHABET);

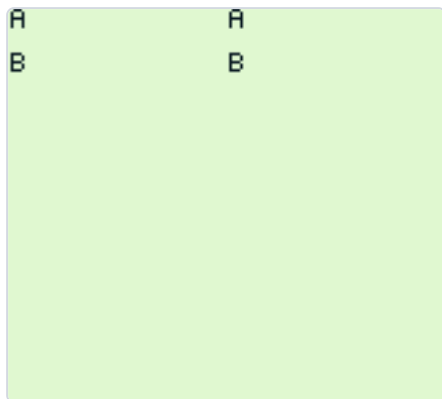
        poser(0, 0, ALPHABET[lente]);           // ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]);           // ligne 2 : la rapide
        poser(10, 0, ALPHABET[lente]);           // la lente, en colonne 10 (le 0.15.1)
        poser(10, 2, ALPHABET[rapide]);         // la rapide, en colonne 10 (le 0.15.1)
    }
}
```

3. Ce qui se passe, étape par étape

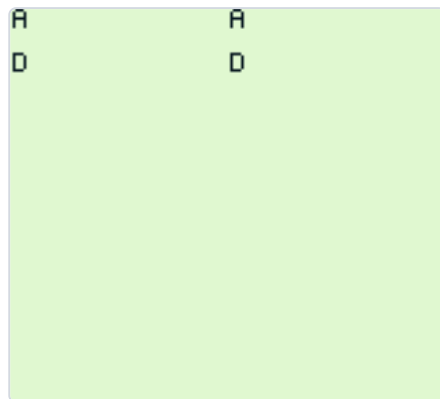
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



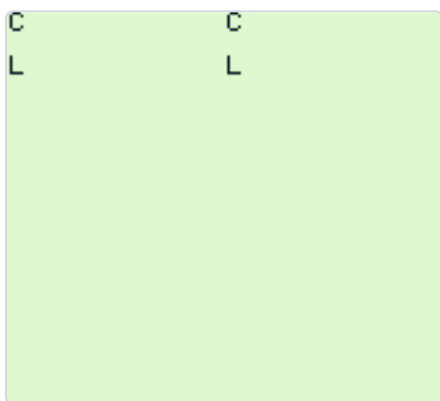
la toute première image — la console vient de démarrer — image 1



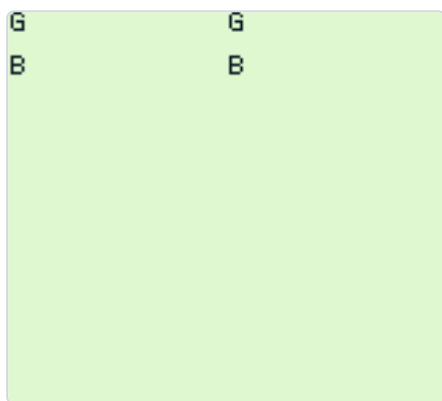
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

La lente et la rapide, en colonne 0 et en colonne 10.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ colonne 0 : la lente (ligne 0) et la rapide (ligne 2) changent
- ✓ colonne 10 : la lente (ligne 0) et la rapide (ligne 2) changent

Ensuite : **0.16. Deux rythmes, écrits en millisecondes** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++